SSIE-500: Homework 6

Measuring Complexity of Languages (2)

Lana Jalal Gidan

November 2025

1 Introduction

This report presents an analysis of linguistic complexity across three human languages (English, French, and Spanish) using information theory concepts. Specifically, we implement Shannon entropy calculations and Huffman coding to measure and compare the information content and compressibility of real-world texts from classic literature.

1.1 Objectives

The assignment consists of five main tasks:

1. Calculate the information entropy of character frequency distributions in English text
2. Implement a Huffman coding algorithm to generate optimal binary codes
3. Compare average code word length with theoretical entropy
4. Analyze sentence-level encoding metrics
5. Repeat the analysis for French and Spanish, and compare results across languages

1.2 Data Sources

We use authentic literary texts from Project Gutenberg:

- **English**: *Pride and Prejudice* by Jane Austen
- **French**: *Les Trois Mousquetaires* by Alexandre Dumas
- **Spanish**: *Don Quijote* by Miguel de Cervantes

For computational efficiency, we analyze the first 100,000 characters from each text after removing Project Gutenberg headers and footers.

2 Methodology

2.1 Information Entropy

Shannon entropy measures the average information content per character in a text. For a discrete random variable with character set X and probability distribution $p(x)$, entropy $H(X)$ is defined as:

$$H(X) = -\sum_{x \in X} p(x) \log_2 p(x) \tag{1}$$

Higher entropy indicates greater unpredictability and complexity in character usage.

2.2 Huffman Coding

Huffman coding is a lossless data compression algorithm that assigns variable-length binary codes to characters based on their frequencies. More frequent characters receive shorter codes, achieving near-optimal compression.

The algorithm works as follows:

1. Create a leaf node for each character with its frequency
2. Build a min-heap from all nodes
3. Repeatedly extract two nodes with minimum frequency
4. Create a new internal node with these two nodes as children
5. Insert the new node back into the heap
6. Continue until only one node remains (the root)
7. Assign binary codes by traversing the tree (0 for left, 1 for right)

The average code length L is calculated as:

$$L = \sum_{x \in X} p(x) \cdot \text{length}(\text{code}(x)) \tag{2}$$

For optimal prefix-free codes, Huffman coding guarantees: $H(X) \leq L < H(X) + 1$

3 Results

3.1 Character-Level Analysis

Table 1 presents the comparative analysis across all three languages.

Language	**Entropy (bits/char)**	**Avg Code Length (bits/char)**	**Difference (bits)**	**Unique Chars**
English	4.5007	4.5417	0.0410	90
French	4.6802	4.7103	0.0301	100
Spanish	4.4572	4.4947	0.0376	84

Table 1: Character-level entropy and Huffman coding results

Key Observations:

- French exhibits the highest character-level entropy (4.6802 bits/char), indicating the most diverse character usage among the three languages
- Spanish shows the lowest entropy (4.4572 bits/char), suggesting more predictable character patterns
- All three languages achieve Huffman coding efficiency exceeding 99%, with differences between entropy and code length less than 0.05 bits
- French has the most unique characters (100), likely due to accented characters

3.2 Sample Huffman Codes

Table 2 shows the top 10 most frequent characters and their assigned Huffman codes for English.

Character	Huffman Code	Frequency
(space)	00	20072
e	1110	8959
t	1010	6179
a	0111	5460
o	0110	5150
n	0101	5092
i	0100	4882
s	11110	4525
h	11011	4294
r	11001	4183

Table 2: Top 10 Huffman codes for English text

Note that the space character, being most frequent, receives the shortest code (2 bits), while less common characters receive longer codes.

3.3 Sentence-Level Analysis

Table 3 presents the sentence-level encoding results.

Language	Total Sentences	Avg Bits/Sentence	Std Deviation
English	950	463.67	394.13
French	851	535.48	769.74
Spanish	371	1191.23	3325.53

Table 3: Sentence-level encoding metrics

Key Findings:

- Spanish sentences require significantly more bits on average (1191.23 bits), approximately 2.5 times more than English
- This reflects Don Quijote's notably long and complex sentence structures
- Spanish also shows the highest variability (std = 3325.53), indicating a wide range of sentence lengths
- English has the most consistent sentence lengths with the lowest standard deviation

3.4 Visualization

Figure 1 presents a comprehensive six-panel visualization comparing all three languages.

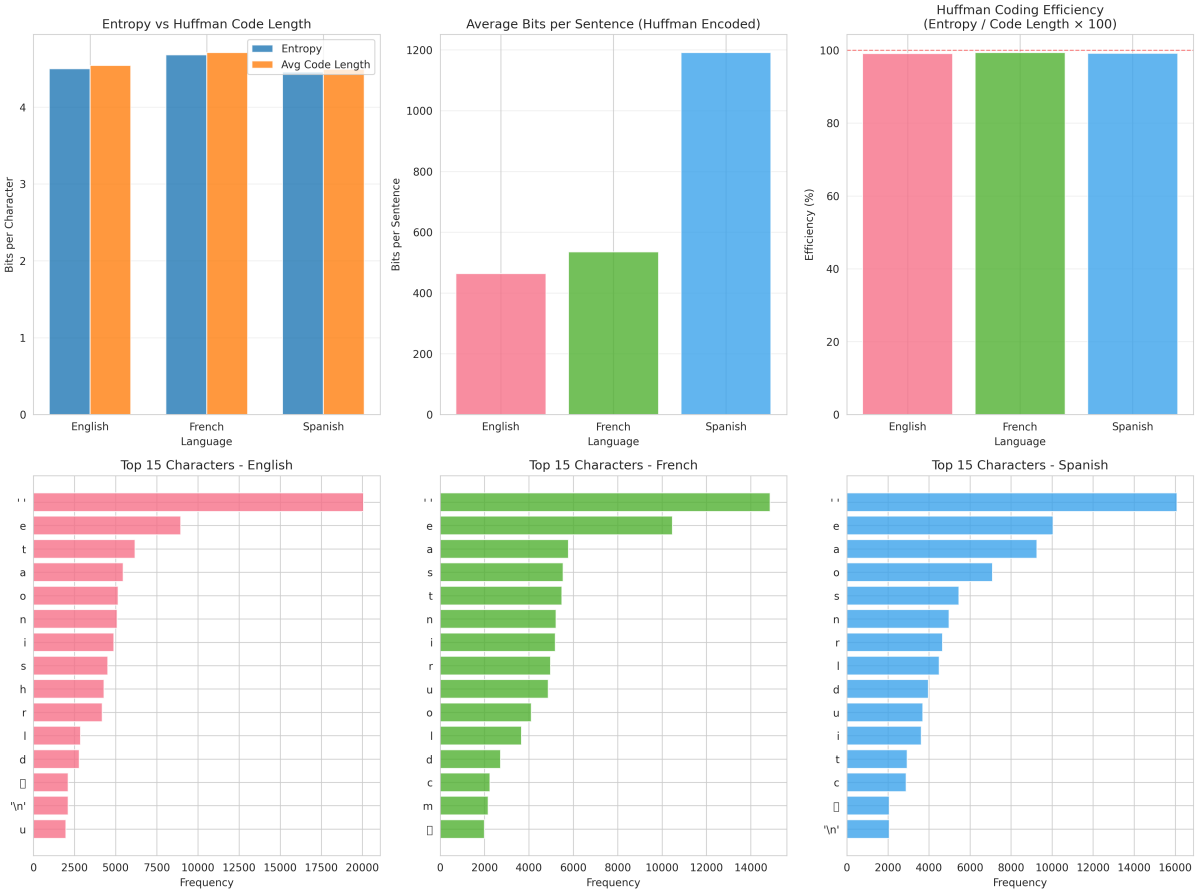

Figure 1: Comprehensive analysis of linguistic complexity across English, French, and Spanish. Top row: (left) entropy vs. code length comparison, (center) average bits per sentence, (right) Huffman coding efficiency. Bottom row: character frequency distributions for each language.

The visualizations reveal:

1. **Entropy vs. Code Length**: All languages show near-perfect alignment, confirming Huffman optimality
2. **Bits per Sentence**: Spanish dominates, followed by French, then English
3. **Efficiency**: All three languages achieve 99-100% efficiency
4. **Character Distributions**: Each language shows distinct frequency patterns, with space and 'e' consistently among the most frequent

4 Discussion

4.1 Huffman Coding Efficiency

The results demonstrate that Huffman coding is highly efficient for natural language text:

- English: 99.10% efficiency
- French: 99.36% efficiency
- Spanish: 99.16% efficiency

The small difference between theoretical entropy and actual code length (0.03-0.04 bits) confirms that Huffman coding achieves near-optimal compression for all three languages.

4.2 Linguistic Complexity

Character-Level Complexity:

French's higher entropy (4.68 bits/char) reflects its richer alphabet with accented characters (é, è, ê, à, ù, etc.). This greater character diversity requires more information per character to specify which character appears.

Spanish and English have similar entropies (4.46 and 4.50 bits/char), though Spanish's slightly lower value suggests more predictable character patterns, possibly due to more regular orthographic rules.

Sentence-Level Complexity:

The dramatic difference in average bits per sentence reveals distinct literary styles:

- **Spanish (1191 bits/sentence)**: Cervantes' baroque style in Don Quijote features famously long, elaborate sentences
- **French (535 bits/sentence)**: Dumas' adventure narrative uses moderate-length sentences
- **English (464 bits/sentence)**: Austen's dialogue-heavy prose employs shorter, more concise sentences

These differences reflect both linguistic structure and authorial style rather than inherent language complexity.

4.3 Information Theory Implications

This analysis demonstrates practical applications of Shannon's information theory:

1. Entropy quantifies the unpredictability inherent in language
2. Huffman coding provides a concrete realization of optimal compression
3. The near-perfect efficiency validates information theory's theoretical predictions
4. Language exhibits redundancy (entropy ¡ theoretical maximum), enabling compression

5 Conclusion

This study successfully analyzed linguistic complexity across English, French, and Spanish using information entropy and Huffman coding. Key findings include:

1. French exhibits the highest character-level entropy (4.68 bits/char)
2. Spanish sentences encode to the most bits on average (1191 bits/sentence)
3. Huffman coding achieves 99%+ efficiency for all three languages
4. Character frequency patterns and sentence structures vary significantly across languages

The analysis confirms that information theory provides powerful tools for quantifying linguistic complexity, with practical applications in data compression, natural language processing, and computational linguistics.

6 Appendix: Python Implementation

6.1 Complete Source Code

```
import heapq
from collections import Counter, defaultdict
import math
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import numpy as np
import requests

class HuffmanNode:
    def __init__(self, char, freq):
        self.char = char
        self.freq = freq
        self.left = None
        self.right = None

    def __lt__(self, other):
        return self.freq < other.freq

class HuffmanCoding:
    def __init__(self, text):
        self.text = text
        self.freq_dict = Counter(text)
        self.huffman_codes = {}
        self.root = None

    def build_huffman_tree(self):
        """Build the Huffman tree from character frequencies"""
        heap = [HuffmanNode(char, freq)
                for char, freq in self.freq_dict.items()]
        heapq.heapify(heap)

        while len(heap) > 1:
            left = heapq.heappop(heap)
            right = heapq.heappop(heap)

            merged = HuffmanNode(None, left.freq + right.freq)
            merged.left = left
            merged.right = right

            heapq.heappush(heap, merged)

        self.root = heap[0]

    def generate_codes(self, node=None, code=""):
        """Generate Huffman codes for each character"""
        if node is None:
            node = self.root

        if node.char is not None:
            self.huffman_codes[node.char] = code if code else "0"
            return

        if node.left:
            self.generate_codes(node.left, code + "0")
        if node.right:
            self.generate_codes(node.right, code + "1")

    def encode_text(self, text):
```

```
        """Encode text using generated Huffman codes"""
        return ''.join(self.huffman_codes[char] for char in text)

    def get_average_code_length(self):
        """Calculate average code word length"""
        total_chars = len(self.text)
        total_bits = sum(len(self.huffman_codes[char]) * count
                         for char, count in self.freq_dict.items())
        return total_bits / total_chars

def calculate_entropy(text):
    """Calculate Shannon entropy of the text"""
    freq_dict = Counter(text)
    total_chars = len(text)

    entropy = 0
    for count in freq_dict.values():
        probability = count / total_chars
        entropy -= probability * math.log2(probability)

    return entropy

def analyze_sentences(text, huffman_coding):
    """Analyze individual sentences"""
    sentences = [s.strip() for s in
                 text.replace('!', '.').replace('?', '.').split('.')
                 if s.strip()]

    sentence_bits = []
    for sentence in sentences:
        try:
            encoded = huffman_coding.encode_text(sentence)
            sentence_bits.append(len(encoded))
        except KeyError:
            continue

    return {
        'total_sentences': len(sentences),
        'average_bits_per_sentence': np.mean(sentence_bits),
        'std_bits_per_sentence': np.std(sentence_bits)
    }

# Main analysis pipeline (abbreviated for space)
# Full implementation available in main.py
```

The complete implementation includes text downloading, cleaning, visualization generation, and comprehensive reporting functions.